Activity 4 Report: Database Users, Roles, Privileges and Access Control

COMP 018 Database Administration | Project tested: CampusSafe (Campus Hazard and Safety Reporting System) (PHP + PostgreSQL) | Date of testing: 7 November 2026

1. Cover and links

Item	Value
Testing group	G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph
Project tested	CampusSafe (Campus Hazard and Safety Reporting System)
Owning group	Group 7 (member names are not listed in the project files)
Repository	grp07-campussafe (project zip grp07-campussafe-main; the repository link is not included in the files we received)
Tracker	https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests
Evidence files	a4_access.txt, a3_deploy.txt

Every result below comes from this run on a freshly created database.

2. Access-control plan (from the project)

database/03_roles_and_privileges.sql creates three group roles (app_runtime_grp, app_readonly_grp, app_backup_grp, all NOLOGIN) and three login accounts: campussafe_app (the web app, runtime group), campussafe_report (read-only reporting) and campussafe_backup (backups). It revokes the PUBLIC connect grant and CREATE on schema public, grants CONNECT and USAGE to the groups, and sets default privileges for tables created later. The application roles are Administrator, Maintenance, Student, Faculty/Staff and Auditor, checked in PHP with Auth::requireRole().

3. The roles script, run as shipped

Check	Result
First run	Ran with no errors. The script creates a role only when it is missing (DO block)
Second run	No errors, only notices that the memberships already exist. The script is repeatable
Role attributes	The three accounts: LOGIN, not superuser, no CREATEDB, no CREATEROLE, no replication, SCRAM-SHA-256 passwords. The groups: NOLOGIN
app_runtime_grp (S I U D T)	app_user SIU, assignment SIUD, attachment SI-D, campus_location SIU, hazard_category SIU, hazard_report SIU, maintenance_action SI, report_status_history SI, role S. No TRUNCATE anywhere and no DELETE on reports
app_readonly_grp	SELECT on 7 tables. No access to app_user (so no password hashes) or role
app_backup_grp	SELECT on all 9 tables and all sequences

Schema and database	USAGE on public, no CREATE. CONNECT for the three groups, PUBLIC has no CONNECT
Passwords	The script sets the sample password change_me_locally for all three accounts

One thing to know when testing on a shared server: the role names app_runtime_grp, app_readonly_grp and app_backup_grp are generic. On this server other projects already used them (their login accounts are members of the same groups), so the script reused the existing groups instead of creating them. Two projects sharing one server would share these groups.

4. Tests with a real login

Test	campussafe_app	campussafe_report	campussafe_backup
SELECT hazard_report	Allowed	Allowed	Allowed
SELECT app_user (including password_hash)	Allowed	Denied	Allowed
INSERT a category	Allowed	Denied	Denied
UPDATE app_user (role_id)	Allowed	Denied	Denied
DELETE hazard_report	Denied	Denied	Denied
DELETE attachment	Allowed (0 rows matched)	Denied	Denied
TRUNCATE hazard_report	Denied	Denied	Denied
CREATE TABLE	Denied	Denied	Denied
CREATE ROLE, read pg_authid	Denied	Denied	Denied
VACUUM	Skipped, warning	Skipped, warning	Skipped, warning
Connect to the postgres database	Denied	Denied	Denied

The access-control design matches its plan. The one point to look at is that campussafe_app may update any column of app_user, including role_id and password_hash. The application needs to edit users, but a column-level grant (full_name, email, account_status, role_id, password_hash) would at least name the columns on purpose.

5. The application running as campussafe_app

Step	Result
Student logs in and submits a report	Works (report #21)
Administrator opens dashboard, verify queue, user list and statistics	Work
Administrator verifies report #21	Works. The report is Verified (Medium)
Administrator creates a user (a4_user) and a category	Both work
Technician flow as campussafe_app	Already covered in the postgres run (Activity 3); the technician page also loads as campussafe_app

Unlike the other projects in this series, the shipped grants are enough for the whole application. No extra grant was needed.

6. Application roles against pages

Role	Allowed	Blocked (HTTP 403)
Administrator	Home, list, new report, verify, assign, users, categories, locations, statistics	Maintenance pages

Maintenance	Home, list, new report, work queue, assigned	All admin pages
Student	Home, list, new report	All admin and maintenance pages
Faculty/Staff	Home, list, new report	All admin and maintenance pages
Auditor	Home, list, new report, statistics	Every other admin page and maintenance pages

The checks were done on the server (Auth::requireRole), so hiding a menu link is not the only barrier. The page checks were run twice, once with the app running as postgres and once as campussafe_app, with the same results for the pages tested both times.

7. Other access checks

Check	Result
Pages without login	All redirect to the login page
Session after logout	The old cookie is redirected to the login page (HTTP 302)
Post without CSRF token (student, report form)	HTTP 403. The CSRF token is checked
Six wrong passwords, then the right one	All six answered "Invalid username or password" and the right one then logged in. No lockout or delay
Passwords at rest	All 7 users have bcrypt hashes
Secrets in Git	.env is in .gitignore. .env.example holds the sample database password and the demo password
pg_hba.conf	scram-sha-256 on every rule

8. Issues and recommendations

#	Finding	Recommendation
1	campussafe_app can update any column of app_user, including role_id and password_hash	Use column-level UPDATE grants and keep user administration in one small code path
2	Generic group names (app_runtime_grp and others) collide with other projects on a shared server	Prefix the groups with the project, for example campussafe_runtime_grp
3	No lockout after repeated wrong passwords	Add a delay or a limit
4	The sample passwords (change_me_locally and Password123!) are used for every account	Generate passwords during setup
5	install.php stays open after setup (Activity 3)	Remove it or refuse to run once installed

9. Feedback for the owners

This is the most complete access-control design of the projects we tested: three login accounts with different jobs, no DELETE on reports, no TRUNCATE, a read-only reporting account that cannot see password hashes, a repeatable script, and the app works with it. Findings 1 and 2 are the main improvements.

10. Limitations

The role names were shared with other projects on the same server, as described above.

AI usage declaration

- Tool: Claude (Anthropic).
- Tasks: interpreting the results of the testing.
- Validation: the commands were run by the group on its own computer, and every result in this report comes from the saved output files.